



Materiały dydaktyczne

Przedmiot: **Programowanie obiektowe/Pracownia programowania obiektowego**

Opracował: **Mirosław Miciak**

Ćwiczenie: **C7**

Tematy: Podział programu na funkcje, Biblioteki języka programowania, Proste i złożone typy danych, Zmienne, Operacje na zmiennych: wejścia, wyjścia, arytmetyczne logiczne, Operacje na typach złożonych, Operatory arytmetyczne, przypisania, porównania, Operatory logiczne i bitowe, Operatory do obsługi łańcuchów, Instrukcja warunkowa i instrukcja wyboru, Instrukcje pętli, Wykorzystanie wybranej biblioteki matematycznej, Wykorzystanie bibliotek algorytmów, Funkcje, Klasa, obiekt, metoda, pole, Projektowanie prostych klas, Zmienne obiektowe, Zakres widoczności pól klasy, Metody klasy, Konstruktor i destruktor, Konstruktor kopiujący, Metody klasy - zakres widoczności, Prosta aplikacja obiektowa: konstruktor, destruktor, **Składniki statyczne klasy, Klasy dziedziczone, Klasy zaprzyjaźnione, Dziedziczenie, Hermetyzacja, Polimorfizm, Funkcje zaprzyjaźnione z klasą, Metody do obsługi elementów statycznych klasy, Klasy bazowe i pochodne, Metody wirtualne, Klasy abstrakcyjne, Szablony klas dla prostych typów liczbowych, Obsługa wyjątków, Instrukcja throw, Obsługa błędów wykonania aplikacji.**

Składniki statyczne klasy

Składniki statyczne klasy odnoszą się do elementów, które należą do samej klasy, a nie do jej instancji. Statyczne pola są współdzielone przez wszystkie obiekty danej klasy, co oznacza, że każde z tych pól ma jedną wspólną wartość dla wszystkich instancji. Statyczne metody mogą być wywoływane bez tworzenia obiektu danej klasy.

```
class Bank
{
    public static double oprocentowanieKonta = 3.5; // Statyczne pole
    public string nazwaKonta;

    public Bank(string nazwa)
    {
        nazwaKonta = nazwa;
    }

    public void PokazSzczegolyKonta()
    {
        Console.WriteLine($"Konto: {nazwaKonta}, Oprocentowanie: {oprocentowanieKonta}%");
    }

    public static void ZmienOprocentowanie(double noweOprocentowanie)
    {
        oprocentowanieKonta = noweOprocentowanie;
    }
}

class Program
{
    static void Main(string[] args)
    {
        Bank konto1 = new Bank("Osobiste");
        Bank konto2 = new Bank("Firmowe");

        konto1.PokazSzczegolyKonta();
    }
}
```

```

        konto2.PokazSzczegolyKonta();

        // Zmiana oprocentowania przez metodę statyczną
        Bank.ZmienOprocentowanie(4.2);

        konto1.PokazSzczegolyKonta();
        konto2.PokazSzczegolyKonta();
    }
}

```

W tym przykładzie mamy klasę Bank ze statycznym polem oprocentowanieKonta. Wartość tego pola zmienia się dla wszystkich obiektów za pomocą statycznej metody ZmienOprocentowanie.

Dziedziczenie

Dziedziczenie pozwala tworzyć nowe klasy na podstawie istniejących klas, co umożliwia ponowne wykorzystanie kodu. Klasa bazowa dostarcza pola i metody, które mogą być rozszerzane przez klasy pochodne. Dziedziczenie jest podstawą wielokrotnego wykorzystania kodu w programowaniu obiektowym.

```

class Pojazd
{
    public string Marka { get; set; }
    public int RokProdukcji { get; set; }

    public Pojazd(string marka, int rok)
    {
        Marka = marka;
        RokProdukcji = rok;
    }

    public void WyszwietlInformacje()
    {
        Console.WriteLine($"Marka: {Marka}, Rok produkcji: {RokProdukcji}");
    }

    public void Jedz()
    {
        Console.WriteLine("Pojazd jedzie");
    }
}

class Samochod : Pojazd
{
    public int LiczbaDrzwi { get; set; }

    public Samochod(string marka, int rok, int liczbaDrzwi) : base(marka, rok)
    {
        LiczbaDrzwi = liczbaDrzwi;
    }

    public void Klakson()
    {
        Console.WriteLine("Samochód używa klaksonu");
    }

    public void WyszwietlInformacje()
    {
        base.WyszwietlInformacje();
        Console.WriteLine($"Liczba drzwi: {LiczbaDrzwi}");
    }
}

class Program
{
    static void Main(string[] args)
    {
        Samochod mojSamochod = new Samochod("Toyota", 2021, 5);
        mojSamochod.WyszwietlInformacje();
        mojSamochod.Jedz();
        mojSamochod.Klakson();
    }
}

```

```
}
}
```

W powyższym przykładzie klasa Samochod dziedziczy po klasie Pojazd, dzięki czemu ma dostęp do jej pól i metod. Klasa Samochod rozszerza funkcjonalność klasy bazowej, dodając nowe pola i metody. Następny przykładowy program w C#, który tworzy obiekt klasy Pixel, ustawia wartości pól x i y z klasy Punkt oraz kolor z klasy Pixel, a następnie wywołuje odpowiednie metody do wyświetlenia tych wartości zarówno z klasy bazowej (Punkt), jak i z klasy dziedziczącej (Pixel).

```
using System;

// Definicja nadklasy Punkt
class Punkt
{
    // Pola x i y
    protected int x, y;

    // Konstruktor nadklasy Punkt
    public Punkt()
    {
        Console.WriteLine("Konstruktor nadklasy Punkt");
    }

    // Metoda do ustawiania wartości pól x i y
    public void UstawXY(int x, int y)
    {
        this.x = x;
        this.y = y;
    }

    // Metoda do wyświetlania wartości x i y
    public void PokazXY()
    {
        Console.WriteLine("x: " + x + " y: " + y);
    }
}

// Definicja podklasy Pixel, która dziedziczy po klasie Punkt
class Pixel : Punkt
{
    // Dodatkowe pole kolor
    public string kolor;

    // Konstruktor podklasy Pixel
    public Pixel() : base() // Wywołanie konstruktora nadklasy
    {
        Console.WriteLine("Konstruktor podklasy Pixel");
    }

    // Metoda do wyświetlania wartości x, y oraz kolor
    public void PokazPixel()
    {
        Console.WriteLine("x: " + x + " y: " + y + " kolor: " + kolor);
    }
}

// Klasa Program, w której znajduje się metoda Main
class Program
{
    static void Main(string[] args)
    {
        // Tworzenie obiektu klasy Pixel
        Pixel ob = new Pixel();

        // Ustawianie wartości pól x, y klasy Punkt
        ob.UstawXY(7, 8);

        // Ustawianie wartości pola kolor klasy Pixel
        ob.kolor = "zielony";

        // Wyświetlanie danych z klasy nadklasy Punkt
        Console.WriteLine("Dane obiektu nadklasy Punkt: ");
    }
}
```



```

        ob.PokazXY();

        Console.WriteLine();

        // Wyświetlanie danych z klasy podklasy Pixel
        Console.WriteLine("Dane obiektu podklasy Pixel: ");
        ob.PokazPixel();
    }
}

```

Opis programu:

Klasa Punkt

Zawiera pola x i y, które są chronione (protected), aby były dostępne w klasach dziedziczących.

Zawiera metodę UstawXY(int x, int y), która ustawia wartości pól x i y.

Metoda PokazXY() wyświetla wartości pól x i y.

Klasa Pixel

Dziedziczy po klasie Punkt.

Posiada dodatkowe pole kolor.

Metoda PokazPixel() wyświetla wartości pól x, y oraz kolor.

Konstruktor klasy Pixel wywołuje konstruktor klasy bazowej (Punkt) przy pomocy base().

Klasa Program

W metodzie Main() tworzony jest obiekt klasy Pixel.

Za pomocą metody UstawXY() ustawiane są wartości pól x i y odziedziczone po klasie Punkt.

Pole kolor jest ustawiane bezpośrednio w klasie Pixel.

Wywołanie metod PokazXY() (z klasy Punkt) i PokazPixel() (z klasy Pixel) wyświetla odpowiednie wartości.

W następnym przykładzie klasa B dziedziczy po klasie A, a metoda Pokaz w klasie B przykrywa metodę Pokaz z klasy A.

```

// Klasa A (bazowa)
class A
{
    // Prywatne pole
    private int i;

    // Konstruktor klasy A z jednym parametrem
    public A(int a)
    {
        i = a;
    }

    // Wirtualna metoda, którą można nadpisać w podklasie
    public virtual void Pokaz()
    {
        Console.WriteLine("i: " + i);
    }
}

// Klasa B dziedzicząca po klasie A
class B : A
{
    // Pole w klasie B
    private int k;

    // Konstruktor klasy B z dwoma parametrami, wywołujący konstruktor klasy A
    public B(int a, int b) : base(a)
    {
        k = b;
    }
}

```

```

    }

    // Nadpisanie metody Pokaz z klasy A
    public override void Pokaz()
    {
        Console.WriteLine("k: " + k);
    }
}

// Klasa Program
class Program
{
    static void Main(string[] args)
    {
        // Tworzenie obiektu klasy B
        B ob = new B(23, 34);

        // Wywołanie metody Pokaz dla obiektu klasy B
        ob.Pokaz();
    }
}

```

Opis programu

Klasa A

Ma prywatne pole i, które jest inicjowane przez konstruktor z jednym parametrem.

Metoda Pokaz() jest wirtualna (virtual), co oznacza, że może być nadpisana w klasach dziedziczących.

Metoda ta wyświetla wartość pola i.

Klasa B (dziedziczy po klasie A)

Ma dodatkowe prywatne pole k, które jest inicjowane przez konstruktor z dwoma parametrami.

Konstruktor wywołuje konstruktor klasy bazowej (base(a)), aby zainicjować pole i w klasie A.

Nadpisuje metodę Pokaz() z klasy A przy użyciu słowa kluczowego override. W tej wersji metoda wyświetla wartość pola k.

Klasa Program

Tworzy obiekt klasy B i wywołuje metodę Pokaz(). Dzięki nadpisaniu metody, wywoływana jest wersja z klasy B, która wyświetla wartość k.

Klasy zaprzyjaźnione

W C# nie istnieje bezpośrednia koncepcja klas zaprzyjaźnionych znana z C++. Jednak można osiągnąć podobny efekt poprzez umieszczanie klas wewnątrz innych klas, dając im dostęp do prywatnych składników zewnętrznej klasy.

```

class Dom
{
    private string adres = "Warszawa, ul. Wiosenna 15";

    public class Okno
    {
        public void PokazAdres(Dom dom)
        {
            Console.WriteLine($"Adres domu to: {dom.adres}");
        }
    }
}

class Program
{
    static void Main(string[] args)
    {
        Dom dom = new Dom();
        Dom.Okno okno = new Dom.Okno();
    }
}

```

```
        okno.PokazAdres(dom);  
    }  
}
```

W tym przykładzie klasa wewnętrzna Okno ma dostęp do prywatnego pola adres klasy zewnętrznej Dom dzięki temu, że znajduje się wewnątrz tej samej klasy.

Polimorfizm i dziedziczenie

Polimorfizm w C# pozwala na korzystanie z metod klasy bazowej przez klasy pochodne, z możliwością ich nadpisania (ang. override). Aby metoda mogła być nadpisana, musi być zadeklarowana jako virtual w klasie bazowej.

```
class Zwierze  
{  
    public virtual void DajGlos()  
    {  
        Console.WriteLine("Zwierze wydaje dźwięk");  
    }  
}  
  
class Kot : Zwierze  
{  
    public override void DajGlos()  
    {  
        Console.WriteLine("Miau");  
    }  
}  
  
class Pies : Zwierze  
{  
    public override void DajGlos()  
    {  
        Console.WriteLine("Hau hau");  
    }  
}  
  
class Program  
{  
    static void Main(string[] args)  
    {  
        Zwierze mojeZwierze = new Kot();  
        mojeZwierze.DajGlos();  
  
        mojeZwierze = new Pies();  
        mojeZwierze.DajGlos();  
    }  
}
```

W tym przykładzie klasy Kot i Pies nadpisują metodę DajGlos z klasy Zwierze, co umożliwia różne zachowanie w zależności od typu obiektu, mimo używania referencji klasy bazowej.

Zadania do samodzielnego rozwiązania

1. Składniki statyczne klasy
Stwórz klasę Student, która będzie posiadała statyczne pole liczbaStudentow, zliczające utworzone obiekty klasy Student. Dodaj konstruktor, który zwiększa wartość liczbaStudentow, oraz metodę statyczną, która wyświetla tę wartość.
2. Składniki statyczne klasy
Stwórz klasę Piłka, która będzie posiadała metodę pokazującą liczbę wystąpień obiektów typu Piłka.
3. Dziedziczenie
Stwórz klasę Zwierze z metodą DajGlos. Następnie stwórz klasę Pies, która dziedziczy po klasie Zwierze i nadpisuje metodę DajGlos, aby wyświetlać "Hau hau".
4. Dziedziczenie

Stwórz nową podklasę Pixel3D (dziedzicząca po Pixel), dodaj nowe pole z, które reprezentuje trzeci wymiar. Utwórz metodę PokazPixel3D(), która wyświetla wartości x, y, z oraz kolor.

5. Dziedziczenie

Dodaj nową podklasę C, dziedziczącą po klasie B. Klasa C będzie miała dodatkowe pole m, a metoda Pokaz() zostanie nadpisana, aby wyświetlała zarówno wartość k (z klasy B), jak i nową wartość m.

6. Klasy zaprzyjaźnione

Stwórz klasę Dom z prywatnym polem adres. Następnie stwórz klasę wewnętrzną Okno, która ma metodę wyświetlającą wartość pola adres z klasy Dom.

7. Polimorfizm i dziedziczenie

Stwórz klasę Instrument z metodą Zagraj. Nadpisz tę metodę w klasach Gitara i Pianino, każda z nich powinna wyświetlać dźwięk charakterystyczny dla danego instrumentu.